



GUÍA DE ACTIVIDAD PRÁCTICA

Arquitectura y Diseño de Software · Unidad 3 · Diseña TechShop Cloud antes de escribir una línea de código

Programa	Especialización en Ingeniería de Software · Posgrado	Asignatura	Arquitectura y Diseño de Software
Unidad	Unidad 3 · Práctica opcional	Unidad / Corte	3 · Corte 1
Modalidad	En equipo (el mismo del proyecto)	Periodo	2026-B
Tipo	Formativa (opcional, sin nota)	Entrega	Aula Moodle, según indique el tutor

Objetivos

- Ubicar cada elemento de TechShop Cloud en la capa hexagonal que le corresponde.
- Definir el comportamiento del sistema ante el fallo de cada dependencia externa.
- Construir la matriz de permisos de los tres roles con mínimo privilegio.
- Repartir el trabajo del equipo en un orden que no genere bloqueos.

1. Objetivo

Dejar decidido, antes de programar, lo que después es caro de cambiar: **las capas, los fallos y los permisos** de TechShop Cloud. Práctica **opcional y no calificable**, preparación directa de la **Actividad 3 · Caso de estudio en equipos (40 %)**.



info: Los tres bloques de esta práctica corresponden al 30 %, 15 % y 15 % de la rúbrica del proyecto: arquitectura, seguridad y resiliencia. Lo que decidas aquí es literalmente lo que se evalúa allá.

2. El caso

TechShop Cloud es una plataforma de comercio electrónico de productos tecnológicos que debe soportar picos de tráfico en campañas.

Requisito funcional	
RF-01	Gestión de productos con categorías y búsqueda
RF-02	Carrito de compras con persistencia
RF-03	Checkout con varios métodos de pago
RF-04	Gestión de usuarios y perfiles
RF-05	Inventario en tiempo real
RF-06	Notificaciones de estado del pedido

Requisito no funcional	Valor
RNF-01 · Escalabilidad	Crecimiento horizontal y vertical automático
RNF-02 · Rendimiento	< 200 ms en el 95 % de las peticiones
RNF-03 · Disponibilidad	99,9 %
RNF-04 · Tolerancia a fallos	Recuperación automática
RNF-05 · Despliegue	Docker y Kubernetes
RNF-06 · Monitoreo	Métricas en tiempo real y alertas

3. Enunciado

Parte A — Ubica cada pieza en su capa

Clasifica los siguientes elementos en las cuatro capas de la arquitectura hexagonal. Hay trampas deliberadas.



Elemento	Capa
Product con el método decreaseStock()	?
ProductUseCase	?
ProductRepository (la interfaz)	?
JpaProductRepository (la implementación)	?
ProductController (REST)	?
JwtTokenProvider	?
La regla «no se puede vender sin inventario»	?
El cliente HTTP de la pasarela de pagos	?
La anotación @Entity de JPA	?
CircuitBreakerConfig	?

Capas disponibles: **dominio, aplicación, infraestructura, interfaz.**

warn: Las dos últimas filas son la trampa principal. Si @Entity termina sobre la entidad de dominio, el dominio pasa a depender de JPA y se rompe la regla de dependencia. Decide si lo aceptas —es una concesión común— y **escribe por qué**; lo que no se puede es no darse cuenta.

Parte B — Qué pasa cuando algo falla

Para cada dependencia externa, decide el comportamiento. No basta con decir «se aplica circuit breaker»: hay que decir **qué recibe el usuario**.

Dependencia	Falla en...	Timeout	Reintento	¿Fallback?	Qué ve el usuario
Pasarela de pagos	Checkout				
Servicio de notificaciones	Tras el pedido				
Base de datos	Cualquier operación				
Caché (Redis)	Consulta de				





		catálogo				
Servicio de inventario	de	Agregar al carrito				

tip: Tres de estas cinco admiten **degradación elegante**: el sistema puede seguir funcionando peor en lugar de no funcionar. Identifica cuáles, porque es exactamente lo que distingue una propuesta de nivel de posgrado de una lista de patrones copiada.

Parte C — Matriz de permisos

Completa la matriz para los tres roles del enunciado, aplicando **mínimo privilegio**:

Operación	ADMIN	PROVIDER	USER
Crear producto			
Editar producto propio			
Editar producto ajeno			
Ver catálogo			
Agregar al carrito			
Hacer checkout			
Ver pedidos propios			
Ver pedidos de otros			
Cambiar rol de un usuario			

Y responde: ¿cuáles de estas operaciones no se pueden resolver solo con el rol?

warn: «Editar producto propio» no se decide con el rol: se decide comparando el dueño del producto con el usuario del token. Si esa comprobación falta, tienes un control de acceso roto —el primer riesgo de la lista OWASP— aunque el @PreAuthorize esté puesto.



Parte D — Plan del equipo

El proyecto es en equipo. Reparte el trabajo evitando que unos esperen a otros:

Fase	Qué se hace	Quién	Depende de
1	Dominio + pruebas (TDD)		—
2	Puertos y casos de uso		fase 1
3	Adaptador REST		fase 2
4	Adaptador de persistencia		fase 2
5	Seguridad (JWT + roles)		fase 3
6	Resiliencia + observabilidad		fase 3
7	Docker + Kubernetes		fases 3 y 4

info: Las fases 3 y 4 son paralelas **porque los puertos ya están definidos**. Esa es la ventaja práctica de la arquitectura hexagonal en un trabajo en equipo: dos personas pueden avanzar contra la misma interfaz sin pisarse.

4. Lista de verificación

- ☐ Los diez elementos de la Parte A están clasificados.
- ☐ La decisión sobre @Entity en el dominio está escrita y justificada.
- ☐ Las cinco dependencias tienen comportamiento definido ante fallo.
- ☐ Al menos tres tienen degradación elegante, no solo error.
- ☐ Cada fallback dice **qué ve el usuario**, no solo qué hace el sistema.
- ☐ La matriz de permisos está completa.
- ☐ Están identificadas las operaciones que requieren comprobar pertenencia.



- [] El plan de equipo tiene responsable por fase y dependencias explícitas.

5. Entrega

- Documento **PDF de 2 a 3 páginas** con las cuatro partes.
- Nombre sugerido: U3-practica-<equipo>.pdf.
- Conviene hacerla **en equipo**: es la reunión de diseño del proyecto.

6. Rúbrica orientativa (para autoevaluarte)

Criterio	Logrado	En proceso	Insuficiente
Capas	Clasificación correcta y concesiones justificadas	Correcta sin justificar las dudosas	Mezcla dominio con infraestructura
Fallos	Comportamiento y experiencia de usuario definidos	Patrones nombrados sin consecuencia	«Se usa circuit breaker» y nada más
Permisos	Matriz completa y pertenencia identificada	Matriz completa sin detectar la pertenencia	Todo resuelto con roles
Plan	Fases paralelas bien identificadas	Plan secuencial que bloquea al equipo	Sin reparto

7. Si quieres ir más allá

- Calcula el presupuesto de caída del **99,9 %** y contrástalo con el tiempo que tardaría tu equipo en restaurar a mano. La diferencia es el argumento de la redundancia.
- Escribe el **ADR** de una decisión de esta práctica —por ejemplo, permitir @Entity en el dominio o separar el modelo de persistencia— con sus alternativas descartadas.



CORHUILA

CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL - SNIES 2828

tip: Ese ADR es reutilizable tal cual en la documentación del proyecto, que pesa 10 % de la rúbrica y suele escribirse a las carreras el último día.

📍 Sede Quirinal: Calle 21 No. 6 - 01
📍 Sede Prado Alto: Calle 8 No. 32 - 49 PBX: (608) 8754220
📍 Sede Pitalito: Carrera 2 No. 1 - 27 - PBX: (608) 8350459
✉ Email: contacto@corhuila.edu.co - www.corhuila.edu.co
Personería Jurídica Res. Ministerio de Educación No. 21000 de Diciembre 22 de 1989
NIT. 800.107584-2



CORPORACIÓN UNIVERSITARIA DEL HUILA - CORHUILA
"Diseño y prestación de servicios de docencia, investigación y extensión de programas de pregrado y posgrado, en sus sedes Neiva y Pitalito"